

NotMapped Attribute in Entity Framework Core (EF Core)

In this article, I will discuss **NotMapped Data Annotation Attribute in Entity Framework Core** (EF Core) with Examples. Please read our previous article, discussing [InverseProperty Attribute in Entity Framework Core](#) with Examples.

What is the [NotMapped] Attribute in Entity Framework Core?

The [NotMapped] attribute in Entity Framework Core (EF Core) that a particular property or class should not be mapped to a database table or column. EF Core will ignore this property or class when generating the database schema and performing operations like inserts, updates, or queries.

This attribute is part of the **System.ComponentModel.DataAnnotations.Schema** namespace. It plays an important role in scenarios where properties in our entity classes are used for calculations, temporary data storage, or logic that doesn't require database persistence.

If you go to the definition of NotMappedAttribute class, you will see the following. As you can see, this class has one parameterless constructor.

```
namespace System.ComponentModel.DataAnnotations.Schema
{
    public class NotMappedAttribute : Attribute
    {
        public NotMappedAttribute();
    }
}
```

NotMapped Attribute of Model Properties Real-Time Example: Employee Management System

Imagine we are developing an Employee Management System for a company. We have an Employee entity that stores details about each employee, such as their first name, last name, date of joining, date of birth, and other information.

The company wants to display each employee's Age (current age based on the date of birth), Tenure (calculates how long the employee has been with the company based on the Date of Joining), and full name (combination of first name and last name) in the application.

Since these values can change over time, they should be calculated on the fly based on the existing data and not stored in the database. We can use the NotMapped attribute for this purpose.

Employee Entity

Create a class file named **Employee.cs** within the **Entities** folder, and then copy and paste the following code. The Tenure property calculates the number of years the employee has worked in the company. The FullName property is the combination of First and Last Names, and the Age property is calculated based on the Date of Birth value. These three properties will not be stored in the database, so we marked them with the NotMapped Attribute.

```
using System.ComponentModel.DataAnnotations.Schema;
namespace EFCoreCodeFirstDemo.Entities
{
```

```

public class Employee
{
    public int EmployeeId { get; set; } //PK
    public string FirstName { get; set; }
    public string LastName { get; set; }
    public DateTime DateOfJoining { get; set; }
    public DateTime DateOfBirth { get; set; }
    // Navigation property for related Department
    public int DepartmentId { get; set; } //FK
    public Department Department { get; set; }
    // Dynamically calculated property
    [NotMapped]
    public int Tenure
    {
        get
        {
            var today = DateTime.Today;
            int years = today.Year - DateOfJoining.Year;
            // Adjust if the employee's anniversary date hasn't occurred
yet this year
            if (DateOfJoining.Date > today.AddYears(-years))
                years--;
            return years;
        }
    }
    [NotMapped]
    public int Age
    {
        get
        {
            var today = DateTime.Today;
            var age = today.Year - DateOfBirth.Year;
            // Adjust age if the birthday hasn't occurred yet this year
            if (DateOfBirth.Date > today.AddYears(-age))
                age--;
            return age;
        }
    }
    [NotMapped]
    public string FullName => $"{FirstName} {LastName}";
}
}

```

Explanation

- **[NotMapped] Attribute:** Applied to the Tenure, Age, and FullName properties to indicate that EF Core should not map them to columns in the database.
- **Age Property:** Calculates the employee's age based on DateOfBirth and the current date.
- **FullName Property:** Concatenates FirstName and LastName to provide the employee's full name.
- **Tenure Property:** Calculates the number of years the employee has been working at the company by subtracting the year of DateOfJoining from the current year.

Department Entity

Create a class file named **Department.cs** within the **Entities** folder and then copy and paste the following code. This will represent the Department entity. There are also one-to-many relationships between Departments and Employees, i.e., one department can have many employees.

```
namespace EFCoreCodeFirstDemo.Entities
{
    public class Department
    {
        public int DepartmentId { get; set; }
        public string Name { get; set; }
        // Navigation property for related Employees
        public ICollection<Employee> Employees { get; set; }
    }
}
```

DbContext Class

The DbContext class manages database operations. Modify the EFCoreDbContext class as follows. Here, we are adding some seed data to test the NotMapped attribute.

```
using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo.Entities
{
    public class EFCoreDbContext : DbContext
    {
        protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
        {
            optionsBuilder.UseSqlServer(@"Server=LAPTOP-6P5NK25R\SQLSERVER2022DEV;Database=EmployeesDB;Trusted_Connection=True;TrustServerCertificate=True");
        }
        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            // Seeding Department data
            modelBuilder.Entity<Department>().HasData(
                new Department { DepartmentId = 1, Name = "Human Resources" },
                new Department { DepartmentId = 2, Name = "IT" }
            );
            // Seeding Employee data
            modelBuilder.Entity<Employee>().HasData(
                new Employee
                {
                    EmployeeId = 1,
                    FirstName = "Hina",
                    LastName = "Sharma",
                    DateOfJoining = new DateTime(2015, 6, 1),
                    DateOfBirth = new DateTime(1990, 5, 15),
                    DepartmentId = 1 // Human Resources
                },
                new Employee
            );
        }
    }
}
```

```

    {
        EmployeeId = 2,
        FirstName = "Pranaya",
        LastName = "Rout",
        DateOfJoining = new DateTime(2018, 4, 10),
        DateOfBirth = new DateTime(1992, 8, 20),
        DepartmentId = 2 // IT
    },
    new Employee
    {
        EmployeeId = 3,
        FirstName = "Rakesh",
        LastName = "Singh",
        DateOfJoining = new DateTime(2020, 3, 15),
        DateOfBirth = new DateTime(1985, 12, 5),
        DepartmentId = 2 // IT
    },
    new Employee
    {
        EmployeeId = 4,
        FirstName = "Priyanka",
        LastName = "Tiwary",
        DateOfJoining = new DateTime(2017, 11, 20),
        DateOfBirth = new DateTime(1995, 3, 10),
        DepartmentId = 1 // Human Resources
    }
    );
}

public DbSet<Department> Departments { get; set; }
public DbSet<Employee> Employees { get; set; }
}
}

```

Generating Migration and Syncing with Database:

Now, open the Package Manager Console and execute the **Add-Migration** and **Update-Database** commands to generate the Migration file and apply the pending migration to sync the database with our Models.

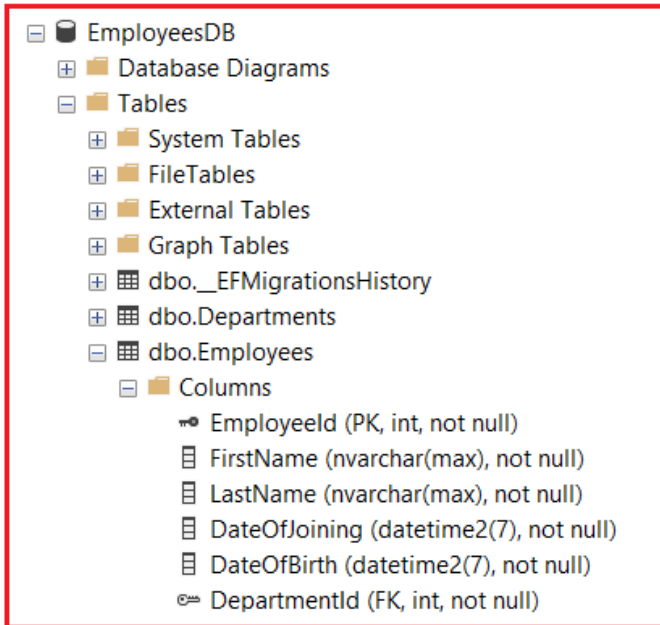


```

Package Manager Console
Package source: All [v] Default project: EFCoreCodeFirstDemo [v]
PM> add-migration DBMigration1
Build started...
Build succeeded.
To undo this action, use Remove-Migration.
PM> update-database
Build started...
Build succeeded.
Applying migration '20230809123849_DBMigration1'.
Done.

```

If you verify the database, you will only see the EmployeesDB database with the required Employees and Departments table. You can also verify that there is no column mapping for the Age, FullName, and Tenure properties, as shown in the image below.



Modifying Program Class:

Next, modify the Program class as follows. Here, we fetch and display all the Departments and their employees. We also display the Age, Tenure, and Full Name. When we retrieve an Employee from the database and access the Tenure, Age, or Full Name property, these values are calculated dynamically based on other properties.

```
using EFCoreCodeFirstDemo.Entities;
using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo
{
    internal class Program
    {
        static void Main(string[] args)
        {
            using var context = new EFCoreDbContext();
            // Fetch and display departments with their employees
            var departments = context.Departments.Include(d =>
d.Employees).ToList();
            Console.WriteLine("Departments and Employees:");
            foreach (var department in departments)
            {
                Console.WriteLine($"Department: {department.Name}");
                foreach (var employee in department.Employees)
                {
                    Console.WriteLine($"  \tEmployee: {employee.FullName}");
                    Console.WriteLine($"  \tTenure: {employee.Tenure} years and
Date of Joining: {employee.DateOfJoining:yyyy-MM-dd}");
                    Console.WriteLine($"  \tAge: {employee.Age} years and Date
of Birth: {employee.DateOfBirth:yyyy-MM-dd}");
                    Console.WriteLine(); //Line Break
                }
            }
        }
    }
}
```

```

    }
  }
}

```

Output:

```

Departments and Employees:
Department: Human Resources
    Employee: Hina Sharma
    Tenure: 9 years and Date of Joining: 2015-06-01
    Age: 34 years and Date of Birth: 1990-05-15

    Employee: Priyanka Tiwary
    Tenure: 6 years and Date of Joining: 2017-11-20
    Age: 29 years and Date of Birth: 1995-03-10

Department: IT
    Employee: Pranaya Rout
    Tenure: 6 years and Date of Joining: 2018-04-10
    Age: 32 years and Date of Birth: 1992-08-20

    Employee: Rakesh Singh
    Tenure: 4 years and Date of Joining: 2020-03-15
    Age: 38 years and Date of Birth: 1985-12-05

```

Benefits of Using NotMapped Attribute on Model Properties:

When an HR manager views the details of an employee, the application fetches all the employee's information, including the calculated tenure, age, and full name. These properties dynamically calculate the values based on the employee's date of joining, date of birth, and first and last name, so they always reflect the current value.

Why NotMapped is Useful Here:

- **Up-to-date Value:** Since Tenure and Age change over time (sometimes we also need to update the name if a wrong name is entered), we want it to be accurate without updating the database yearly. Calculating it dynamically ensures it is always correct.
- **Avoid Redundant Data:** Storing Tenure, Age, and Full Name as columns in the database would require us to update the values periodically, adding complexity to our system and increasing the chance for data inconsistency. Also, storing additional columns means additional storage space is required; hence, the cost also increases.

Example Without Using the [NotMapped] Attribute:

Entity Framework Core will not create a column in the database table for a property if it does not have both a getter and a setter. EF Core effectively ignores properties without a setter or a getter because EF Core can't persist or retrieve values for such properties.

Now, from the Employee entity, we will remove the [NotMapped] attributes and make the properties that don't need to be persisted by only defining the get accessor without a set. This will ensure EF Core ignores these properties when creating the table schema. So, modify the Employee entity as follows:

```

using System.ComponentModel.DataAnnotations.Schema;
namespace EFCoreCodeFirstDemo.Entities
{
    public class Employee
    {
        public int EmployeeId { get; set; } //PK
        public string FirstName { get; set; }
        public string LastName { get; set; }
        public DateTime DateOfJoining { get; set; }
        public DateTime DateOfBirth { get; set; }
        // Navigation property for related Department
        public int DepartmentId { get; set; } //FK
        public Department Department { get; set; }
        // Dynamically calculated property
        public int Tenure
        {
            get
            {
                var today = DateTime.Today;
                int years = today.Year - DateOfJoining.Year;
                // Adjust if the employee's anniversary date hasn't occurred
yet this year
                if (DateOfJoining.Date > today.AddYears(-years))
                    years--;
                return years;
            }
        }
        public int Age
        {
            get
            {
                var today = DateTime.Today;
                var age = today.Year - DateOfBirth.Year;
                // Adjust age if the birthday hasn't occurred yet this year
                if (DateOfBirth.Date > today.AddYears(-age))
                    age--;
                return age;
            }
        }
        public string FullName => $"{FirstName} {LastName}";
    }
}

```

Now, generate and apply the Migration again. It should have created the database without the Tenure, Age, and Full Name columns.

[NotMapped] Attribute with Entity Class in Entity Framework Core

Let's consider a real-time example involving a Financial Management System. We want to apply the NotMapped attribute to an entire entity class to avoid mapping it to a database table. This is a common scenario where we need to

represent a specific type of data that is used purely for business logic, calculations, or data transformations but is not directly stored in the database.

Real-Time Scenario: Financial Report Generation

Imagine we are building a Financial Management System for a company. We have an Expense entity that stores expense details in the database, and we also need to generate a summary report of expenses for different departments within a specific timeframe.

To achieve this, we can create a DepartmentExpenseReport entity with the calculated summary data. This report is used only for business logic purposes, and there's no need to persist this data in the database. Let us proceed and see how we can use the NotMapped attribute to exclude the DepartmentExpenseReport class from database mapping.

Expense Entity (Mapped to Database)

Create a class file named **Expense.cs** within the Entities folder and then copy and paste the following code. The Expense entity represents individual expense records for various departments and is persisted in the database.

```
namespace EFCoreCodeFirstDemo.Entities
{
    public class Expense
    {
        public int ExpenseId { get; set; }
        public string Department { get; set; }
        public string Purpose { get; set; }
        public decimal Amount { get; set; }
        public DateTime Date { get; set; }
    }
}
```

DepartmentExpenseReport Entity (Not Mapped to Database)

Create a class file named **DepartmentExpenseReport.cs** within the Entities folder, and then copy and paste the following code.

```
using System.ComponentModel.DataAnnotations.Schema;
namespace EFCoreCodeFirstDemo.Entities
{
    [NotMapped]
    public class DepartmentExpenseReport
    {
        public string DepartmentName { get; set; }
        public decimal TotalExpenses { get; set; }
        public int NumberOfTransactions { get; set; }
    }
}
```

The DepartmentExpenseReport class is not mapped to any database table because it represents calculated values needed only for business reporting. It uses the [NotMapped] attribute to ensure that Entity Framework ignores this class while generating the database schema, even if we accidentally specified this entity as a DbSet property in the DbContext class.

Benefits of using NotMapped Attribute:

When an admin wants to generate a summary of expenses by department for a specific period, we can create instances of the `DepartmentExpenseReport` class. This summary report contains calculated fields like `TotalExpenses` and `NumberOfTransactions`, which are aggregated from the `Expense` data but do not need to be stored in the database.

Why NotMapped is Useful Here:

- **Avoiding Redundant Data:** The `DepartmentExpenseReport` is a calculated entity, and there's no point in storing this information in the database as it can be generated on the fly whenever required.
- **Business Logic Focused:** This entity is focused on reporting and calculations, not on persistence, making it an ideal candidate for the `[NotMapped]` attribute.

DbContext Class with Seed Data

Please modify the `EFCoreDbContext` class to include the `Expense` entity and seed some initial data. Here, we have also accidentally added the `DepartmentExpenseReport` as a `DbSet` property.

```
using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo.Entities
{
    public class EFCoreDbContext : DbContext
    {
        protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
        {
            optionsBuilder.UseSqlServer(@"Server=LAPTOP-6P5NK25R\SQLSERVER2022DEV;Database=ExpensesDB;Trusted_Connection=True;TrustServerCertificate=True");
        }
        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            // Seed data for the Expense entity
            modelBuilder.Entity<Expense>().HasData(
                new Expense { ExpenseId = 1, Department = "HR", Purpose = "Office Supplies", Amount = 1500, Date = new DateTime(2024, 9, 15) },
                new Expense { ExpenseId = 2, Department = "IT", Purpose = "Software Licenses", Amount = 3500, Date = new DateTime(2024, 9, 16) },
                new Expense { ExpenseId = 3, Department = "IT", Purpose = "Travel Expenses", Amount = 800, Date = new DateTime(2024, 9, 17) },
                new Expense { ExpenseId = 4, Department = "HR", Purpose = "Training", Amount = 1200, Date = new DateTime(2024, 9, 18) },
                new Expense { ExpenseId = 5, Department = "IT", Purpose = "Hardware", Amount = 5500, Date = new DateTime(2024, 9, 19) }
            );
        }
        public DbSet<Expense> Expenses { get; set; }
        public DbSet<DepartmentExpenseReport> DepartmentExpenseReports { get; set; }
    }
}
```

Now, without the Migration process, we can also create the Database, tables, and seed data using the `Database.EnsureCreated()` method of the context object.

Program Class to Generate the Report

Next, please modify the Program class as follows to generate and display the department expense report. Here, **context.Database.EnsureCreated()** method will create the database table and seed the data if the database table has not yet been created.

```
using EFCoreCodeFirstDemo.Entities;
namespace EFCoreCodeFirstDemo
{
    internal class Program
    {
        static void Main(string[] args)
        {
            using (var context = new EFCoreDbContext())
            {
                // Ensure the database is created and seed data is loaded
                context.Database.EnsureCreated();
                // Generate the report
                var report = GenerateDepartmentExpenseReport(context);
                // Display the report
                DisplayExpenseSummary(report);
            }
        }

        public static List<DepartmentExpenseReport>
GenerateDepartmentExpenseReport(EFCoreDbContext context)
        {
            // Group expenses by department and calculate the report data
            var expenseReports = context.Expenses
                .GroupBy(e => e.Department)
                .Select(group => new DepartmentExpenseReport
                {
                    DepartmentName = group.Key,
                    TotalExpenses = group.Sum(e => e.Amount),
                    NumberOfTransactions = group.Count()
                })
                .ToList();
            return expenseReports;
        }

        public static void
DisplayExpenseSummary(List<DepartmentExpenseReport> expenseReports)
        {
            Console.WriteLine("Department Expense Summary Report");
            Console.WriteLine("-----");
            foreach (var report in expenseReports)
            {
                Console.WriteLine($"Department: {report.DepartmentName}");
                Console.WriteLine($"  \tTotal Expenses:
{report.TotalExpenses}");
                Console.WriteLine($"  \tNumber of Transactions:
{report.NumberOfTransactions}");
            }
        }
    }
}
```

```

        Console.WriteLine(); //Line Break
    }
}
}

```

Output:

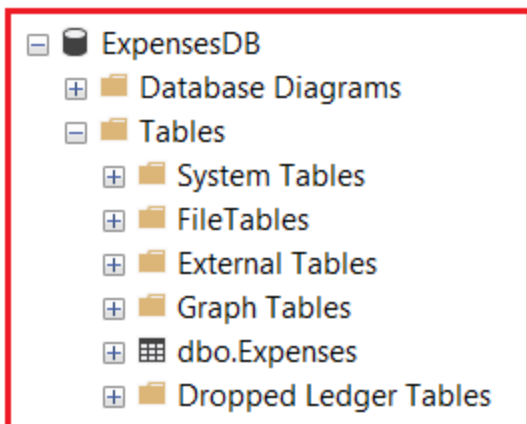
```

Department Expense Summary Report
-----
Department: HR
    Total Expenses: 4000.00
    Number of Transactions: 2

Department: IT
    Total Expenses: 9800.00
    Number of Transactions: 3

```

Now, if you verify the database, then you will see that only the Expenses table is created, as shown in the below image:



Benefits of applying NotMapped Attribute on Entity Level

- **Data Consistency:** The DepartmentExpenseReport always represents up-to-date values because it is calculated dynamically.
- **Reduced Complexity:** There's no need to create extra tables in the database to store temporary or calculated data, reducing the complexity of the database schema.
- **Focused Business Logic:** The NotMapped attribute allows us to focus on business calculations and reporting without worrying about database persistence.;

Usage of [NotMapped] Attribute in EF Core

Applying to a Property: When applied to a property, it ensures that it will not be mapped to a column in the database. This is ideal for properties used solely for internal logic, temporary storage, or calculations that don't need to be stored in the database.

Applying to an Entity Class: The [NotMapped] attribute can also be used at the class level to prevent the entire class from being mapped to a database table. This is useful for scenarios where the class is meant to act as a helper class used only for internal logic, and there is no need to create a table in the database.

In the next article, I will discuss the [Required Attribute in Entity Framework Core](#) with Examples. Here, in this article, I explain the **NotMapped Data Annotation Attribute in Entity Framework Core** with Examples. I hope you enjoyed this NotMapped Attribute in EF Core with Examples article.